

מסמך דרישות תוכנה –
תוכנה לבניית שיבוץ מערכת שעות
גרסה 34.0

A. מבוא

מטרת התוכנה לספק כלי-עזר לתכנון ושיבוץ מערכת הבחינות בהם יבחנו סטודנטים בפקולטה להנדסה. עם הגידול במספר התוכניות ובמספר הקורסים, שיבוץ בחינות שמונע התנגשויות בין בחינות באותו יום או בהפרשים קטנים בין בחינה לבחינה נעשה מורכב יותר ולפעמים לא מתקבל פתרון "אופטימלי".

בגרסה הנוכחית (גרסה $3.0+4.0=34.0$):
נתמך בהוספת דרישות נוספות לגבי שיבוץ בחינות כדי לאפשר מציאת שיבוצים
אופטימליים לפי מספר קריטריונים. וגם – נייצר ערך ללקוח בהתאם להרחבה
כרצונכם.

גרסא זו תהיה מעט ארוכה יותר מהרגיל, ותכלול יותר תכולה מאשר גרסא רגילה. הערכו בהתאם.

B. דרישות מפורטות

1. כללי

1.1. הפונקציונליות החדשה צריכה להיתמך גם בגרסא עם GUI (הרחבה של V2.0) וגם בגרסא מבוססת קבצים (הרחבה של V1.0).

2. קריטריונים נוספים לשיבוץ מערכת בחינות

המשתמש יוכל להגדיר את הקריטריונים הבאים כדרישת סף בנוסף לדרישות שהוגדרו בגרסאות הקודמות. המשמעות של דרישת סף: שיבוץ שלא עומד בתנאי זה נפסל. לכל דרישה שלהלן, ניתן להפעיל אותה או להשאירה כבויה, לפי החלטת המשתמש במסך "הגדרות".
המספר k הוא פרמטר שיקבע על ידי המשתמש ויכול לקבל ערך שונה בכל אחד מהסעיפים הבאים.

- 2.1. מספר הימים בין שתי בחינות בקורסי חובה באותה תוכנית ובאותה שנה לא יהיה קטן מ k כאשר k מספר חיובי שלם. ספירת מספר הימים בסעיף זה ובהמשך כוללת שבתות וחגים.
- 2.2. מספר הימים בין שתי בחינות (כשכל אחת מהבחינות יכולה להיות בקורס חובה או בחירה) באותה תוכנית ובאותה שנה לא יהיה קטן מ k כאשר k מספר חיובי שלם.
- 2.3. מספר ההתנגשויות בין שני קורסי בחירה באותה תוכנית לא יהיה גדול מ k לכל תוכנית כאשר k מספר שלם אי שלילי.

- 2.4. מספר הימים בין הבחינה הראשונה בקורס חובה בתוכנית מסוימת בשנה מסוימת ובמועד נתון לבין הבחינה האחרונה בקורס חובה באותה תוכנית ובאותה שנה ומועד לא יהיה קטן מ k כאשר k מספר חיובי שלם.
- 2.5. מספר הבחינות המקסימלי המשובצות באותו יום לא יהיה גדול מ k כאשר k מספר חיובי שלם.

3. קריטריונים למציאת שיבוץ מיטבי של מערכת בחינות

המשתמש יוכל לבקש למיין את המערכות שעונות על דרישות הסף לפי הקריטריונים הבאים. המשתמש יוכל למיין לפי מספר קריטריונים מהרשימה הבאה, תוך הגדרת הסדר ביניהם (מי עדיף ומי משני). ניתן להניח שבזמן הריצה של מציאת שיבוצים המשתמש עשוי לשנות את הגדרת המיון אך לא את דרישות הסף.

- 3.1. הצגה בסדר יורד של מספר הימים המינימלי שקיים בין שתי בחינות בקורסי חובה באותה תוכנית ובאותה שנה.
- 3.2. הצגה בסדר יורד של ממוצע מספר הימים בין שתי בחינות בקורסי חובה או בחירה באותה תוכנית ובאותה שנה.
- 3.3. הצגה בסדר יורד של מספר ההתנגשויות בין שני קורסי בחירה באותה תוכנית.
- 3.4. הצגה בסדר יורד של מספר הימים בין הבחינה הראשונה בקורס חובה בתוכנית מסוימת בשנה מסוימת ובמועד נתון לבין הבחינה האחרונה בקורס חובה באותה תוכנית ובאותה שנה ומועד.
- 3.5. הצגה בסדר יורד של מספר הבחינות המקסימלי המשובצות באותו יום.

4. הרחבה המייצרת ערך ללקוח

בסעיף זה אתם נדרשים להמציא בעצמכם יכולת חדשה (פיצ'ר - feature) שנותן ערך משמעותי למשתמש. כאן אתם יכולים להיות יצירתיים ולחשוב מעבר לגבולות המשימה על הפיצ'ר האולטימטיבי שחסר במערכת שבניתם. הציון ינתן כפונקציה של מורכבות הפיצ'ר ומועילות הפיצ'ר. רמת המורכבות הצפויה היא בדומה לרמת המורכבות של גרסא 1 או גרסא 2 או פרקים 2+3 לעיל.

(רעיונות לא טובים: חיבור ל AI שמבצע שיבוץ בעצמו; הוספת עוד מסנן או עוד מיון; שינוי המסך שיהיה קריא יותר, וכד')

- 4.1. עליכם להחליט על רעיון הרחבה שעומד בתנאים לעיל
- 4.2. עליכם לכתוב מפרט דרישות מלא המתאר את הפיצ'ר שבניתם, את היכולות הנדרשות ואת התכונות המתאפשרות.
- 4.3. יש להגיש את מפרט הדרישות יחד עם שאר מסמכי ההגשה של גרסא זו

5. עיצוב תוכנה

- 5.1. התכונה תכתב בשפת C++ או פייתון
- 5.2. עיצוב התוכנה יהיה בשיטה מונחת אובייקטים (OOP)

6. ביצועים

- 6.1. מומלץ לאפשר שמירת נתונים פנימית של התוכנה, כלומר, אם לא שינו את קובץ הקורסים/תאריכים, אזי הנתונים ייטענו מקובץ פנימי של התוכנה, ללא צורך בטעינת הנתונים מחדש מהקובץ.
- 6.2. המסכים צריכים להיות תגובתיים. השהיה/"תקיעה" של יותר משניה היא עדות לחוסר תגובתיות.

7. הנחות אחרות, ושונות

- 7.1. פיתוח הקוד יתבצע ב-GIT כאשר לכל חבר צוות חשבון משלו. ניהול הפרויקט יתבצע באפליקציית JIRA.
- 7.2. אחד מחברי הצוות יהיה "מוביל גרסה" ויוביל את קו הפיתוח הראשי, ישבץ משימות ב-JIRA ויהיה אחראי על ענף הקוד הראשי ב-GIT.
- 7.3. פיתוח הגרסה יבוצע בהתאם למתודולוגיית פיתוח AGILE שנלמדה בכיתה ויכלול את המרכיבים הבאים:
- דרישות תוכנה – עבור פיציר ההרחבה שאתם מגדירים, יפורט במפרט דרישות עיצוב התוכנה (יפורט במסמך עיצוב תוכנה, יש להכליל פרק על ממשקי משתמש, התפריטים, הרעיון מאחורי עיצוב הUI/UX ודוגמאות שימוש)
 - תכנון הבדיקות (יפורטו במסמך מפרט בדיקות)
 - מימוש (יפורט ע"י הקוד ב-GIT)
- ביצוע Code-Review (חלקית יתבצע בהשתתפות סגל הקורס. סיכום code review יפורט במסמך סיכום)
- ביצוע הבדיקות על-פי מפרט הבדיקות
- הצגת כלל הפעילות לסגל הקורס.

C. מונחים והסברים נוספים

"התוכנה" – מערכת התוכנה אותה מפתחים

"משתמש" – המשתמש/ת שמפעילה את התוכנה, למשל גורם במזכירות שרוצה לשבץ את כל המבחנים שנלמדים בפקולטה.

"קורס" – לקורס יש מספר מזהה בן 5 ספרות. לכל קורס יש שם מרצה ורשימה של תוכניות לימוד בו הקורס נלמד, כולל מידע אם הקורס חובה או בחירה ובאיזה שנה וסמסטר הוא נלמד באופן רגיל. לכל קורס יש גם מידע על אופן ההערכה (בחינה, פרויקט, השתתפות).

"תקופת בחינות" – טווח תאריכים בהן ניתן לשבץ בחינות בסמסטר נתון ועבור מועד נתון, למשל סמסטר א (FALL) מועדי ב.

"שיבוץ של בחינה" – השמה של בחינה בתאריך אפשרי בתקופת בחינות.

"מערכת בחינות" – שיבוץ אפשרי של הבחינות בכלל הקורסים של התוכניות שנבחרו.

